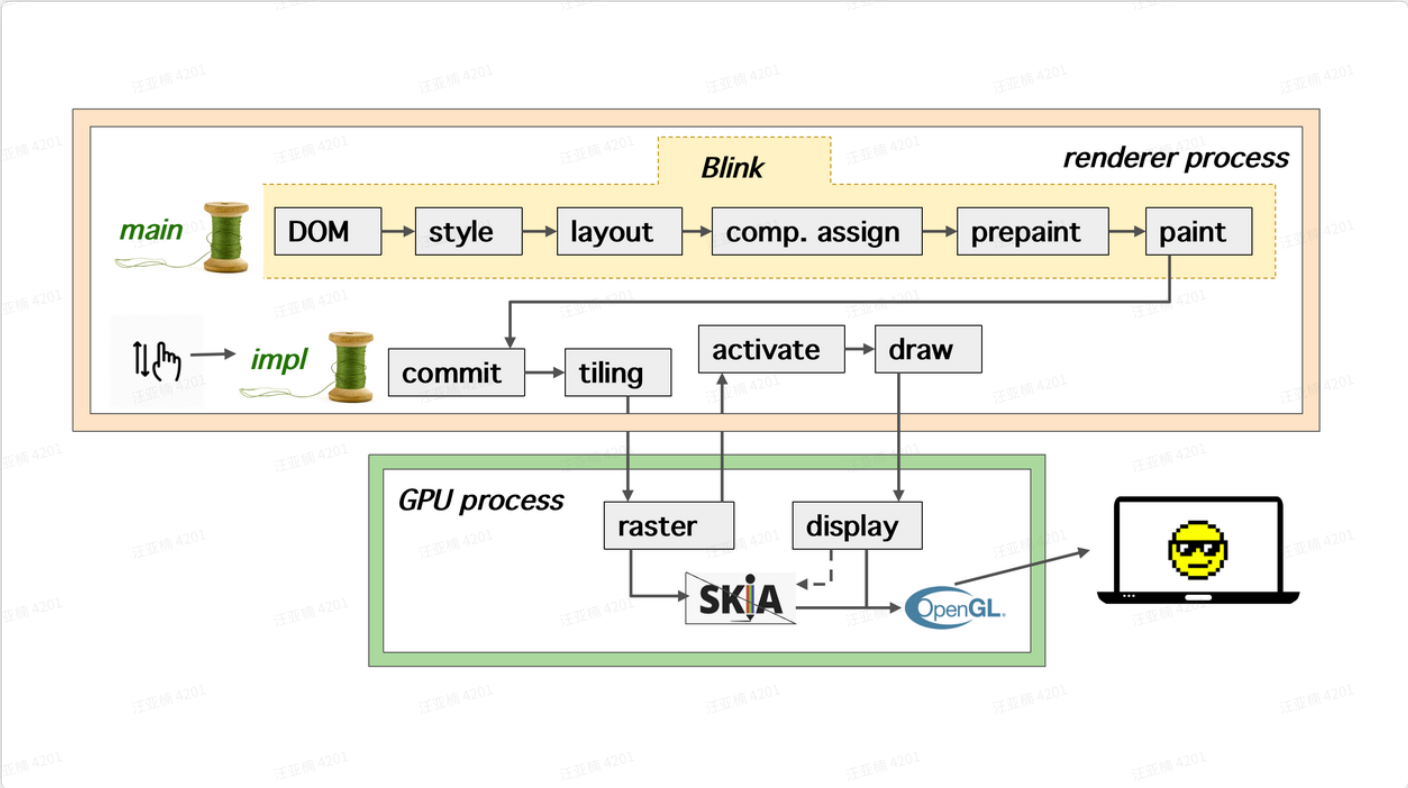
6. Css属性性能优化 / Css Houdini

这节课有两部分内容，第一部分是有助于css性能优化相关的css属性，比如经常被提及的will-change和“硬件加速”，第二部分是对css houdini的一个简单介绍，它是一系列可以被开发者调用的css 渲染引擎api

6.1 浏览器渲染流程回顾

在学习css属性性能优化这一部分内容之前，我们有必要再回顾一下浏览器的渲染原理，下面以谷歌浏览器为例；需要注意的是由于近几年谷歌都在对渲染流程进行重构，部分专有名词或环节可能会发生变化，所以大家在阅读相关文档的时候需要注意一下文档的时间

这里我找到了一份比较详细的流程图，如下：

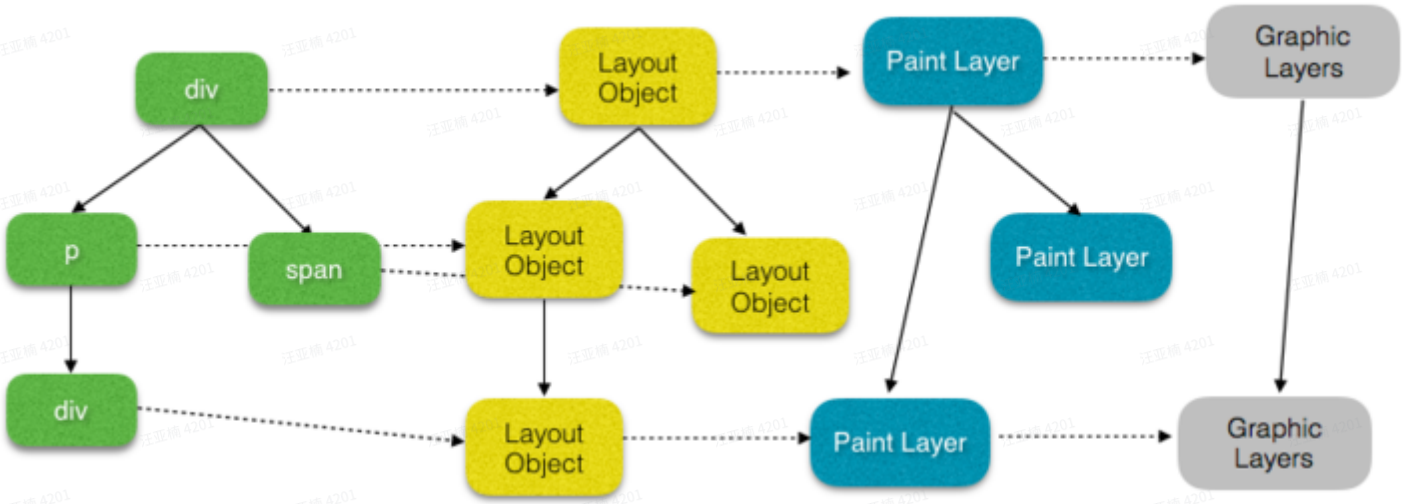


Life of a Pixel

这里的impl是指合成线程 (compositor thread)，渲染流程如下 (load.html)

1. 先是主线程解析文档，生成浏览器能读懂的DOM树结构
2. 解析css，计算样式，生成浏览器浏览器能理解的 `StyleSheets` ，每个DOM节点有其对应的 `ComputedStyle`
3. 布局阶段，计算每个节点在页面上具体的几何位置，该阶段重构后输出为 `NGFragment tree` ，但目前输入输出都混在 `LayoutObjects Tree` 中

4. `compositing assignments` 层提升阶段；页面上的元素，根据所处坐标空间(可以理解为层叠上下文)不同等原因，会被划分为不同的 `PaintLayer` ，而具有 `transform` , `mask` 等属性的 `PaintLayer` 可能会被进一步提升为 `GraphicsLayer` ，没被提升的都属于上级 `GraphicsLayer` 详见



5. Pre-paint 阶段，该阶段有两个任务，一是判断与上一次绘制相比有哪些内容需要被更新，二是构建 `Property Trees`

> Paint invalidation which invalidates display items which need to be painted.
>
> Builds paint property trees.

6. Paint 阶段会依据层叠顺序生成绘制列表，在同一层叠上下文内会先生成同层级所有背景的绘制任务，再生成文字的绘制任务等，等到真正绘制的阶段按列表顺序执行绘制
7. 之后主线程将绘制列表提交到合成线程，合成线程根据绘制列表进行光栅化操作，这一步为了减少资源消耗会对图层进行分块 (tiles)，将图块作为光栅化的基本单位，优先对视口附近的图块进行光栅化，光栅化会生成位图，储存在内存中
8. 光栅化完成后会生成 `draw quads` 命令，该命令会引用光栅化结果，到这一步浏览器进程通过才通过该命令将内容呈现到屏幕上

了解了渲染流程之后我们就能知道为什么css动画比js动画性能好了，关键点就在于层提升，我们还是来看例子(transform.html)

可以看到 js 动画会不断的触发layout

而我们的css动画呢，也触发了layout！但是只有一次，重排数量大大减少。通过这种分层的方式更多的利用GPU来控制样式，减少重排重绘就是所谓的 “硬件加速” ；需要注意的是层提升会加大内存的使用，通过这种方式进行性能优化需要通过开发者工具来检验是否确实产生了更好的效果，均衡使用

但是为什么还是触发了一次重排呢?

这是因为在添加animation属性前，浏览器并不知道元素要执行动画，所以没有进行层提升；当animation属性添加后，才进行层提升，所以导致了一次重排分层

为了让浏览器能在动画前好准备，就需要用到 `will-change`

6.2 will-change

will-change的作用是提前告知浏览器可能变动的属性，让浏览器提前做好准备，提前进行相关计算等，其取值如下

```
will-change: auto | scroll-position | contents | <custom-ident>
```

在该属性出现之前，大家都是通过给元素设置 `translateZ(0)` 这种 hack 的方式提前设置层提升，将可能导致层提升的属性设置为该属性的值，都会使浏览器提前进行层提升

- auto 让浏览器自己猜哪些值会变动
- scroll-position 表示滚动条位置可能发生变化或产生动画
- contents 表示元素内容可能变动或产生动画
- <custom-ident> 表示所有css属性

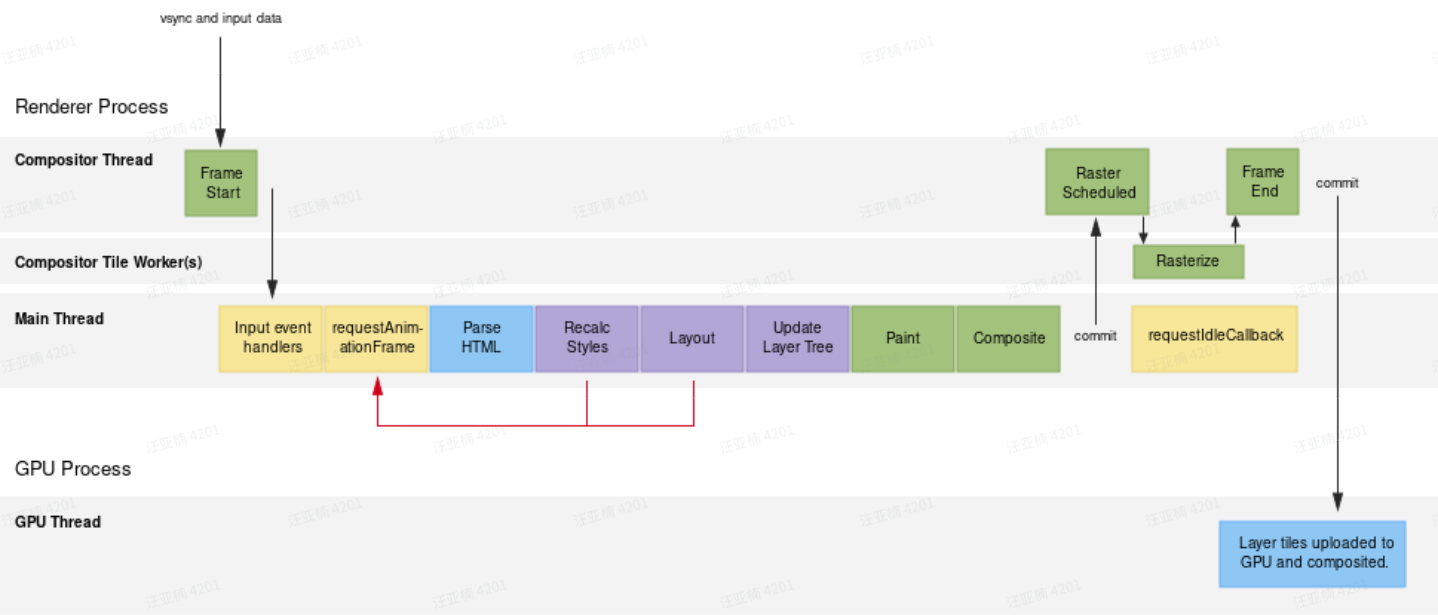
基本上哪里的css属性变化导致了页面的卡顿都可以使用will-change优化，但需要注意的点有：

1. 不要大量使用，该属性会加大内存消耗，大量使用可能起反作用
2. 浏览器本身已经做了十足的性能优化，如果页面本身还没出现性能问题就不要提前使用
3. 由于使用该属性会导致内存消耗增加，应该仅在触发某些条件的时候再为元素添加该属性，同时我们又要为浏览器预留一些准备时间，建议的写法如下

```
.will-change-parent:hover .will-change {
  will-change: transform;
}
.will-change {
  transition: transform 0.3s;
}
.will-change:hover {
  transform: scale(1.5);
}
```

讲到动画我们顺便提一嘴 `requestAnimationFrame` ，它会在每一帧的渲染流程执行前都执行一次，因此使用js实现动画时，相比于 `setInterval` 实际执行时间的不确定性 `requestAnimationFrame` 更加可靠；

而 `requestIdleCallback` 则是在每一帧结束前判断是否有剩余时间，如果有则执行，无则不执行



The Anatomy of a Frame (2016.2)

6.3 contain

contain属性可以让局部的DOM树结构成为一个独立的部分，和页面其他的DOM树结构完全隔离，避免重排重绘影响其余DOM内容，适合用在复杂页面的某个小组件上

其取值如下，可以同时设置多个值

```
contain:  [ size || layout || style || paint ];
contain: content; /* 等同于 contain: layout paint*/
contain: strict; /* 等同于 contain: layout paint size*/
```

- size 表示元素尺寸固定，不受子元素影响
- layout 可以想象成对元素的骨架、框架或者渲染盒子进行了封闭，元素会创建新的层叠上下文，创建新的BFC，形成全新的包含块，影响固定定位等元素偏移
- style 并不是用于限制常规样式，它仅用来限制css计数器相关样式
- paint 与layout类似，并且不会渲染overflow的内容，哪怕设置的是visible

由于现代浏览器的性能已经很好了，所以也是一个比较鸡肋的属性，例

6.4 content-visibility

该属性可以让浏览器决定是否渲染视区以外的元素的内容，对于不渲染的内容浏览器会跳过其 layout 和 painting阶段(相当于设置 display: none), 借此提高页面的渲染性能

该属性兼容性不好，实际使用时问题多多，不推荐使用

6.5 css houdini

它相当于提供一些css引擎的api，让我们可以实现自己的css语法，由于兼容性不太好，且上手成本较高(需要精通 js，css，canvas)，因此这里不介绍具体语法

首先我们来了解一下worklet，它可以让开发者在浏览器渲染线程中的多个部分作为钩子运行，如 `PaintWorklet` ，当变量变动时，它可在paint阶段实时更新渲染，除此之外还有 `AnimationWorklet` 可以用于实现高性能动画，而 `LayoutWorklet` 可以用于实现我们自己的布局规则

我们还可以通过变量的方式定义自己的css属性，比如我们给 `--background-color` 定义了取值类型，继承性和初始值

```
:root {
  --background-color: blue;
}

.box {
  background-color: var(--background-color);
}

@property --background-color {
  syntax: '<color>';
  inherits: false;
  initial-value: blue;
}
```

除此之外，还有用于获取详细的文本对象信息的 `Font Metrics API` ，样式接口合集 `CSS Typed OM` 等